

Computer Programming

Lab report 8

Lab title:

Two-Dimensional Arrays in C++

1. Objective

- To learn the concept of two-dimensional arrays
- To understand declaration and initialization of 2D arrays
- To access elements of two-dimensional arrays
- To use nested loops with arrays
- To input and display matrix elements

2.Introduction

- A two-dimensional array is a collection of one-dimensional arrays. It is used to store data in the form of rows and columns like a table or matrix.

The syntax for declaring a two-dimensional array is:

- `dataType arrayName[rows][columns];`

Example

```
int a[3][4];
```

This array contains:

- 3 rows
- 4 columns

Each element is identified using row and column indexes.

3. Initialization of Two-Dimensional Arrays

- Two-dimensional arrays can be initialized during declaration.

Example

```
int a[3][4] =  
{  
    {0,1,2,3},
```

```
        {4,5,6,7},  
        {8,9,10,11}  
};
```

Another method:

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

4. Accessing Elements of Two-Dimensional Arrays

- Elements are accessed using row and column indexes.

Example

```
a[2][3];
```

- This accesses the 4th element of the 3rd row.

5. Program for Displaying Two-Dimensional Array

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int a[5][2] =  
    {  
        {0,0},  
        {1,2},  
        {2,4},  
        {3,6},  
        {4,8}  
    };  
  
    for(int i = 0; i < 5; i++)  
    {  
        for(int j = 0; j < 2; j++)  
        {  
            cout << "a[" << i << "][" << j << "] : ";  
            cout << a[i][j] << endl;  
        }  
    }  
  
    return 0;  
}
```

6. Program for Assigning Values in Matrix

```
#include <iostream>  
using namespace std;
```

```

int main()
{
    int matrix[5][5];

    for(int m1 = 0; m1 < 5; m1++)
    {
        for(int m2 = 0; m2 < 5; m2++)
        {
            matrix[m1][m2] = 5;
        }
    }

    return 0;
}

```

7. Program for Input and Output of Matrix

```

#include <iostream>
using namespace std;

int main()
{
    int matrix[2][3];

    for(int m1 = 0; m1 < 2; m1++)
    {
        for(int m2 = 0; m2 < 3; m2++)
        {
            cout << "Enter Integer : ";
            cin >> matrix[m1][m2];
        }
    }

    cout << endl;

    for(int m1 = 0; m1 < 2; m1++)
    {
        for(int m2 = 0; m2 < 3; m2++)
        {
            cout << "Your Entered Integer are : ";
            cout << matrix[m1][m2];
            cout << endl;
        }
    }

    return 0;
}

```

Sample Output

Enter Integer : 1
Enter Integer : 2
Enter Integer : 3
Enter Integer : 4
Enter Integer : 5
Enter Integer : 6

Your Entered Integer are : 1
Your Entered Integer are : 2
Your Entered Integer are : 3
Your Entered Integer are : 4
Your Entered Integer are : 5
Your Entered Integer are : 6

8. Result

➤ All programs were compiled and executed successfully.

Task 1: Two-Dimensional Array Declaration and Initialization

Diagram

```
Columns
 0  1  2
+---+---+---+
Row 0 | 1 | 2 | 3 |
+---+---+---+
Row 1 | 4 | 5 | 6 |
+---+---+---+
Row 2 | 7 | 8 | 9 |
+---+---+---+
```

Program

```
#include <iostream>    // Header file for input and output
using namespace std;  // Use standard namespace

int main()
{
    int matrix[3][3] =    // Declare and initialize 3x3 matrix
    {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    for (int i = 0; i < 3; i++)    // Outer loop for rows
    {
        for (int j = 0; j < 3; j++)    // Inner loop for columns
```

```

        {
            cout << matrix[i][j] << " ";    // Display matrix elements
        }

        cout << endl;    // Move to next line
    }

    return 0;    // End program
}

```

Task 2: Sum of Rows and Columns in a Two-Dimensional Array

Diagram

```

+---+---+---+
| 2 | 4 | 6 |
+---+---+---+
| 1 | 3 | 5 |
+---+---+---+
| 7 | 8 | 9 |
+---+---+---+

```

Row Sums:

Row 1 = 12

Row 2 = 9

Row 3 = 24

Column Sums:

Column 1 = 10

Column 2 = 15

Column 3 = 20

Program

```

#include <iostream>    // Header file for input and output
using namespace std;    // Use standard namespace

```

```

int main()
{
    int matrix[3][3] =    // Declare and initialize matrix
    {
        {2, 4, 6},
        {1, 3, 5},
        {7, 8, 9}
    };

    int totalSum = 0;    // Variable for total sum

    for (int i = 0; i < 3; i++)    // Loop for rows
    {

```

```

    int rowSum = 0; // Reset row sum

    for (int j = 0; j < 3; j++) // Loop through columns
    {
        rowSum = rowSum + matrix[i][j]; // Add row elements
    }

    cout << "Sum of Row " << i + 1 << " = " << rowSum << endl; // Display row sum
}

cout << endl; // Blank line

for (int j = 0; j < 3; j++) // Loop for columns
{
    int colSum = 0; // Reset column sum

    for (int i = 0; i < 3; i++) // Loop through rows
    {
        colSum = colSum + matrix[i][j]; // Add column elements
    }

    cout << "Sum of Column " << j + 1 << " = " << colSum << endl; // Display column sum
}

cout << endl; // Blank line

for (int i = 0; i < 3; i++) // Loop for rows
{
    for (int j = 0; j < 3; j++) // Loop for columns
    {
        totalSum = totalSum + matrix[i][j]; // Calculate total sum
    }
}

cout << "Total Sum = " << totalSum << endl; // Display total sum

return 0; // End program
}

```

Task 3: Finding Maximum Value in a Two-Dimensional Array

Diagram

```

+---+---+---+---+
| 12 | 5 | 7 | 9 |
+---+---+---+---+
| 20 | 15 | 3 | 1 |
+---+---+---+---+
| 11 | 18 | 25 | 6 |

```

```

+---+---+---+---+
| 4|10| 8|14|
+---+---+---+---+

```

Maximum Value = 25

Program

```

#include <iostream>    // Header file for input and output
using namespace std;  // Use standard namespace

int main()
{
    int matrix[4][4] =    // Declare and initialize 4x4 matrix
    {
        {12, 5, 7, 9},
        {20, 15, 3, 1},
        {11, 18, 25, 6},
        {4, 10, 8, 14}
    };

    int max = matrix[0][0];    // Assume first element is maximum

    for (int i = 0; i < 4; i++)    // Outer loop for rows
    {
        for (int j = 0; j < 4; j++)    // Inner loop for columns
        {
            if (matrix[i][j] > max)    // Check for greater value
            {
                max = matrix[i][j];    // Update maximum value
            }
        }
    }

    cout << "Maximum Value = " << max << endl;    // Display maximum
    value

    return 0;    // End program
}

```

9. Conclusion

In this lab, we learned:

- The concept of two-dimensional arrays
- Declaration and initialization of matrices

- Accessing matrix elements using indexes
- Using nested loops with arrays
- Taking input and displaying output of matrices

Two-dimensional arrays are useful for handling tabular data and matrix operations in programming.